

SECURITY REVIEW

FOR IZKONEN



INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	PROTOCOL OVERVIEW	04
03	SCOPE	05
04	AUDITORS	06
05	METHODOLOGY	07
06	RISK CLASSIFICATION	08
07	FINDINGS SUMMARY	09
08	DETAILED FINDINGS	10
	L-01 Anti-whale limit computed from MAX_SUPPLY, not current supply	10
	I-01 Three tranche values deviate by one token from a strict Fibonacci series	11
	I-02 Replacement minter must preserve the current tranche index	12
	I-03 startTime_ Can Be Initialized Too Far in the Past	13
	I-04 initialLiquidityMinted is module-local, allowing repeated initial liquidity to break the final tranche	14
	I-05 Old and new minters can remain authorized at the same time	15
09	DISCLAIMER	16
10	ABOUT KANN AUDITS	17

AUDIT SUMMARY

IZKONEN engaged Kann Audits to perform a security review of the IZKONEN smart contracts. From 12 – 14 August, a team of 4 security researchers performed a manual review of the in-scope contracts and additionally triaged findings generated by the Kann AI Security Model. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

6
ISSUES

● CRITICAL	-
● HIGH	-
● MEDIUM	-
● LOW	01
● INFORMATIONAL	05

ENGAGEMENT DETAILS

PREPARED FOR	IZKONEN
PREPARED BY	Kann Audits
DATE REPORTED	15 August 2026
REVIEW PERIOD	12 – 14 August

PROTOCOL OVERVIEW

IZKONEN is an ERC-20 token built on Base with an immutable core and a separate, replaceable emission module. The token has a fixed maximum supply of 38.69 trillion IZK, a yearly Fibonacci-based emission schedule running through 2048, and a one-time initial liquidity allocation. The protocol also includes a 3% max-wallet mechanism, emergency pausing, and governance-controlled roles for managing emissions and protocol configuration.

03 - WHAT WE REVIEWED

SCOPE

REPOSITORY	https://github.com/marin-ctrl/izkonen-audit
AUDITED COMMIT	27bfe39
FINAL COMMIT	7f2137b

#	FILE
01	src/IZKONEN.sol
02	src/IZKONENMinter.sol

04 - THE TEAM ON THIS ENGAGEMENT

AUDITORS

KA

Kann

SECURITY RESEARCHER

VO

Volodymyr

SECURITY RESEARCHER

CO

cosminm53

SECURITY RESEARCHER

ST

Stranger8732

SECURITY RESEARCHER

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01

Kickoff & Context

A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02

Scope & Preparation

Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03

AI First Scan

Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04

Expert Security Review

Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05

Findings & Proofs

Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06

Mitigation Review

We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- ✓ Testing against common and uncommon attack vectors
- ✓ Contract logic matching the client's intended behaviour
- ✓ Manual line-by-line review by experienced auditors
- ✓ Architecture and design analysis for secure component interactions and clear trust boundaries
- ✓ Compliance with industry best practices and known smart contract security standards
- ✓ Cross-referencing patterns with industry-standard implementations
- ✓ Access control verification
- ✓ State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

● CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
● HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
● MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
● LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
● INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

07 - ALL ISSUES

FINDINGS SUMMARY

● CRITICAL —	● HIGH —	● MEDIUM —	● LOW 01	● INFO 05
TOTAL 06 FIXED 02 ACKNOWLEDGED 04				

ID	TITLE	SEVERITY	STATUS
L-01	Anti-whale limit computed from MAX_SUPPLY, not current supply	● LOW	● FIXED
I-01	Three tranche values deviate by one token from a strict Fibonacci series	● INFORMATIONAL	● ACKNOWLEDGED
I-02	Replacement minter must preserve the current tranche index	● INFORMATIONAL	● ACKNOWLEDGED
I-03	startTime_ Can Be Initialized Too Far in the Past	● INFORMATIONAL	● FIXED
I-04	initialLiquidityMinted is module-local, allowing repeated initial liquidity to break the final tranche	● INFORMATIONAL	● ACKNOWLEDGED
I-05	Old and new minters can remain authorized at the same time	● INFORMATIONAL	● ACKNOWLEDGED

DETAILED FINDINGS

L-01

Anti-whale limit computed from MAX_SUPPLY, not current supply

SEVERITY

● LOW

STATUS

● FIXED

■ DESCRIPTION

`MAX_WALLET` is a constant equal to 3% of `MAX_SUPPLY`, or 1,160,700,000,000 IZK. The `_update` function enforces the limit after a transfer by checking whether `balanceOf(to)` exceeds `MAX_WALLET`. However, the public token documentation advertises the limit as “3% of current supply.”

Because the early-year circulating supply is small relative to the fixed maximum-supply-derived limit, the concentration protection is effectively inactive until approximately 2041. During the early emission years, a single wallet can therefore hold a share of the circulating supply many times larger than the advertised 3%.

■ IMPACT

The advertised “maximum 3% per wallet” property is not enforced during the early years of the emission schedule. There is no direct loss of funds, but the implementation does not match the stated tokenomics and provides substantially weaker concentration protection than users may expect.

■ RECOMMENDATION

Calculate the wallet limit dynamically as 3% of `totalSupply()`

■ REMEDIATION

● FIXED

I-01

Three tranche values deviate by one token from a strict Fibonacci series

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

■ DESCRIPTION

The tranche comments and whitepaper language describe a Fibonacci-style emission schedule. The implemented whole-token tranche values mostly follow $\text{tranche}[n] = \text{tranche}[n - 1] + \text{tranche}[n - 2]$, but three entries differ from a strict recurrence by exactly one whole token.

The 2030 tranche is one token above the strict sum, the 2037 tranche is one token below the strict sum, and the 2043 tranche is one token above the strict sum. The discrepancy is economically immaterial at the scale of the schedule and appears to be a rounding or documentation-consistency issue rather than a technical defect. The total schedule plus initial liquidity still reaches the hard cap as tested.

■ IMPACT

The implementation does not exactly match a strict Fibonacci recurrence for every listed year. This may confuse reviewers or users comparing the implementation with the whitepaper, although the discrepancy is limited to one whole token in three tranches.

■ RECOMMENDATION

Update the whitepaper or code comments to state that the tranche table is Fibonacci-derived with three one-token rounding adjustments rather than a strict recurrence for every listed year.

■ REMEDIATION

● ACKNOWLEDGED

I-02

Replacement minter must preserve the current tranche index

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

■ DESCRIPTION

If the Minter contract is replaced, the new instance must be initialized with a `startingYearIndex_` that exactly matches the previous minter's `yearsMinted`. An incorrect or stale index can cause already-minted tranches to be repeated or future tranches to be skipped.

■ IMPACT

Incorrect migration configuration can disrupt the intended emission schedule and may result in extra tokens being minted or scheduled tranches being skipped.

■ RECOMMENDATION

During migration, verify that the replacement minter's `startingYearIndex_` exactly matches the current emission progress before granting it `MINTER_ROLE`.

■ REMEDIATION

● ACKNOWLEDGED

I-03

startTime_ Can Be Initialized Too Far in the Past

SEVERITY

● INFORMATIONAL

STATUS

● FIXED

■ DESCRIPTION

The Minter can be initialized with a `startTime_` that is significantly earlier than the current block timestamp. Without a lower-bound validation, an incorrectly configured deployment can begin with an emission start time that is already far in the past, affecting the intended timing of the schedule.

■ RECOMMENDATION

For a first deployment, add a bounded validation such as:

```
require(
    startTime_ ≤ block.timestamp &&
    startTime_ ≥ block.timestamp - GRACE_PERIOD
);
```

For a replacement minter, derive `startTime_` directly from the retired minter's immutable `startTime` on-chain rather than accepting it as a new external input.

■ REMEDIATION

● FIXED

I-04

initialLiquidityMinted is module-local, allowing repeated initial liquidity to break the final tranche

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

■ DESCRIPTION

The `initialLiquidityMinted` flag is stored in the Minter contract rather than globally in the token. A replacement minter therefore starts with `initialLiquidityMinted = false`, allowing `mintInitialLiquidity()` to be executed again and mint an additional 38.5 billion IZK.

■ IMPACT

A replacement minter can mint the 38.5 billion IZK initial-liquidity allocation a second time, consuming supply reserved for later emissions. When the emission schedule reaches the final tranches, minting can revert because the hard cap has already been consumed.

■ EXECUTION CYCLE

Minter1 executes `mintInitialLiquidity()` legitimately → Minter1 is replaced by Minter2 with `initialLiquidityMinted = false` → Minter2 executes `mintInitialLiquidity()` again → the normal emission schedule continues → at index 22, `token.mint` reverts with `CapExceeded`.

■ RECOMMENDATION

Store the “initial liquidity already minted” state globally in the token, pass the flag explicitly into the replacement minter’s constructor, or otherwise track the already-minted initial-liquidity allocation at the token level.

■ REMEDIATION

● ACKNOWLEDGED

I-05

Old and new minters can remain authorized at the same time

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

■ DESCRIPTION

The token does not maintain a single active minter. If a replacement minter is deployed with the correct `startingYearIndex_` but the previous minter is not deauthorized, both contracts can remain valid minters with the same emission progress. Since `mint()` is permissionless, each open emission window can be executed independently through both modules, causing the same tranche to be emitted twice.

■ IMPACT

Overlapping minters can cause double payouts to the configured market and treasury emission addresses, resulting in over-emission. The hard cap is then reached prematurely, preventing the largest late-stage tranches from being minted. The issue does not allow a third party to redirect the emission to an arbitrary address; the duplicated payouts still go to the configured emission recipients.

■ EXECUTION CYCLE

Minter A is operating → Minter B is added with the correct index → Minter A remains authorized → `mint()` is called on both A and B during each open window → each tranche is emitted twice → in a later year, `token.mint` reverts with `CapExceeded`.

■ RECOMMENDATION

Enforce a single active minter at the token level. For example, store an `activeMinter` address in the token and allow minting only from that address, with replacement performed atomically. Alternatively, track tranche execution directly in the token so the same tranche cannot be minted by more than one module.

■ REMEDIATION

● ACKNOWLEDGED

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.



EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.



AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.



FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.



INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.



KANN AUDITS
WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits